PA 4 参考提示

数据结构课程团队

Contents

CST 4-1-2 Game 2

CST 4-2-2 Pattern Matching 3

CST 4-2-3 Melody 4

CST 4-3-2 Component 5

CST 4-3-3 Sort 6

CST 4-1-2 Game

思路提示

本题的输入为是点权，但是我们可以将点权转移到边权上。具体方法是，将原图中的每条无向边 (u, v) 拆分为两条有向边 $u \to v$ 和 $v \to u$。对于有向边 $u \to v$，其边权设置为终点 v 的点权。这样，一条路径的边权之和就等价于路径上除起点外所有点的点权之和。最终计算总路径长度时，只需在求得的最短路基础上，加上起点的点权即可。

因为所有点权均为正数，转换后的图边权也为正，因此可以使用 Dijkstra 算法来求解单源最短路。由于所有构成最短路径的边会形成一个有向无环图，要统计从起点到终点的最短路径总数，可以在这个 DAG 上进行动态规划进行计数。

难点提示

在使用优先队列优化 Dijkstra 算法时，需快速更新一个节点当前的最短距离。一种方法时根据最短距离不升的特点，直接进行 `percolateUp` 操作（以二叉堆为例）。这需要实时维护每个结点当前在堆中的位置。另一种方式为每次更新都向堆中添加一条最短距离的记录，每次取最小值时检查一下该最短距离是否为实际的最短距离。这种方法不需要实现额外的接口（如 STL 的优先队列没有提供更新接口），但是堆的规模可能会达到 $O(E)$ 级别。

CST 4-2-2　Pattern Matching

思路提示

本题要求我们对二维矩阵进行模式匹配。一种简单的方式便是使用 Rabin-Karp 算法（讲义 13-F2 节）。

首先，计算所有大矩阵中 $m \times n$ 规模的子矩阵的哈希值。这一步形式上可以将 $m \times n$ 的矩阵想象成一个长度为 $m \times n$ 的字符串。需要注意为了避免重复计算，可以类似使用讲义 13-F2 节的方式实现 $O(1)$ 在相邻两个子串间转移。类似于二维前缀和计算，这样的优化也能拓展至二维，细节留给同学自行推导。

在查询时，首先计算目标矩阵的哈希值，随后在大矩阵中统计相同哈希值的数量。这里可以使用一些数据结构优化查询，当然最简单的方法可能是排序 + 二分。

难点提示

由于目标矩阵出现次数可能很多，达到 $O(NM)$ 级别。因此，对每个子矩阵都验证是否与目标矩阵相同的方法可能会超时。一种简单的做法便是当哈希值相同时即认为矩阵相同。这便要求哈希的计算方式确保哈希冲突发生的概率**足够小**，乃至我们可以认为哈希冲突在黑盒测例中几乎不可能存在[1]。

这里我们可以计算对于一个测例，其所有哈希比对均正确的概率。假设哈希函数的值域为 $[0, P)$，则对于两个随机字符串，其发生哈希冲突的概率为 $1/P$。一个测例需要进行 $O(NMk)$ 次字符串比对，这些比对均不能发生哈希冲突，因此可以通过该测例的概率约为 $\left(1 - \frac{1}{P}\right)^{NMk}$。大家可以将使用的 P 代入上式进行评估。上面的分析提示我们，若想降低哈希冲突的概率，哈希函数的值域应足够大，或者等价地，使用多个哈希函数以扩大等效的哈希值域。

此外，一个健壮的哈希函数也有助于减少哈希冲突，详见 09-B3 节。这里推荐使用多项式函数 $F(S) = \left(\sum_{i=0}^{|S|-1} x^i S_i\right) \bmod P$，其中 x 为一个给定的参数，P 为模数（通常采用质数）。这种方法在实现相邻字符串的 $O(1)$ 转移时计算较为简单。

一个经典的“不够健壮”的哈希函数是利用无符号整数计算发生溢出时的截断特性，直接使用无符号整数计算字符串所对应多项式的值。这等价于取模数 $P = 2^w$，其中 w 是计算机的字长（32 或 64）。这种方法可被针对性地构造哈希冲突，对此感兴趣的同学可以在互联网搜索“自然溢出哈希”进行了解。注意 09-B3 节中代码所展示的并**不是**自然溢出哈希。

[1] 这一策略在实际生产中也经常使用，例如传输文件时，可先计算文件的哈希值（例如 SHA-256，此场景亦称其为指纹），若已存在相同指纹的文件则不再传输。

CST 4-2-3　Melody

思路提示

此问题可以借助 KMP 算法中的 `next` 数组来解决。我们观察到，对于任意前缀，其所有的公共前后缀可以通过从该位置开始，沿着 `next` 数组不断向前回溯得到。例如，长度为 `n` 的前缀，其所有公共前后缀的长度构成的序列为 `next[n]`, `next[next[n]]`, `next[next[next[n]]]`, …直到 0。

基于 `next` 数组的这一特性，我们可以构建一棵树。树中的每个节点 `i` 代表字符串中长度为 `i` 的前缀，从节点 `i` 向其父节点 `next[i]` 连边。在这棵树上，一个前缀的所有公共前后缀恰好对应于该前缀节点到根节点的路径上的所有节点。

因此，多个前缀的最长公共前后缀问题，就转化为在失配树上寻找这些前缀对应节点的“最深公共祖先”。这等价于求解它们的最近公共祖先（LCA）。由于字符串是动态增长的，我们需要支持在线构建失配树并查询 LCA。考虑到数据规模，可以采用倍增法等高效算法进行 LCA 的查询。

倍增法的核心思想是记录每个节点的第 2^i 级祖先，在查询时，采用类似二分的方式判断 LCA 的位置。具体算法可在互联网搜索或通过询问 AI 进行学习（提醒不要忘记标注 Honor Code!）。

难点提示

注意处理 `next[k]` $> k/2$ 的情况。

CST 4-3-2　Component

思路提示

计算第 k 大的数，可以转换为计算最大的 k 个数中，最小的那一个。因此可以考虑用一个小顶堆，并且限制它的容量为 k。每当第 $k+1$ 个元素入堆的时候，我们都完成一次出堆。因此堆顶即为第 k 大的数。

合并两个连通分量各自的堆可采用左式堆完成。合并之后的数据量可能大于 k，只需保留最大的 k 个即可。同时为了判断是否需要执行合并操作，我们可以使用并查集判断两个两个点是否在同一连通分量。

难点提示

合并后的剔除操作应如何实现从而保证复杂度正确？

CST 4-3-3　Sort

解题思路

首先根据比较树（讲义 05-E 节），比较次数下界为 $\log_{3!}(n!) \approx n \log_6 n$。实际上我们难以达到这一个下界，但这个式子提示我们应该尽可能增加比较结果的 “可能数量”。例如，若我们在之前的比较中已经得知 $a < b$，那么比较 a, b, c 时只会有 3 种可能的结果，那么采用这种方式比较次数下界就会为 $n \log_3 n$。换言之，我们即需要增加比较能够提供给我们的 “信息量”。

若希望基于归并排序解题，显然二路归并无法完全利用三元比较器的所有信息，因此可以采取更高的归并路数进行。在设计过程中可以思考，每次比较可以选取一个当前最小值，那么应该如何利用剩余的大小信息从而减少后续比较的次数。例如，通过一次比较得知 $a < b < c$ 且 a 为当前最小值并加入至结果。那么除此之外，我们还知道了 b, c 的大小关系，进一步得知下一次比较时 c 不可能为最小值。因此下一次比较就不需要选择 c。

若采用快速排序，那么我们的轴点就需要同时和两个数比较，我们应该采用什么样的方法来保证我们的轴点有比较良好的性质。例如，是否可以利用比较的两个数的大小关系已知的特性。